PROMPT ENGINEERING MASTERY BUCKET · MODULE 11

Model Comparison

Choosing the right model for the job — a practical comparison framework across GPT, Claude, Gemini, Llama, Mistral, Qwen, and DeepSeek: strengths, weaknesses, best use cases, context limits, pricing, and tool support.

DIFFICULTY LEVEL
Beginner–Intermediate

ESTIMATED READING TIME
50–65 minutes

ESTIMATED PRACTICE TIME
70–90 minutes

PREREQUISITES
Modules 01–10

LEARNING OUTCOMES

By the end of this module, you will be able to: describe the general positioning and philosophy of the major LLM families; evaluate models along consistent dimensions (capability, cost, context, tool support); and choose a sensible model for a given task instead of defaulting to habit or hype.

Table of Contents

1. Introduction

What is this topic?

Every technique in this course so far has been model-agnostic on purpose. This module is about the one variable that changes everything else: which model you actually build on. Different model families have different strengths, costs, and philosophies — knowing the landscape helps you make that choice deliberately.

Why is it important?

Picking a model based on hype or habit, rather than fit, is one of the most common and costly mistakes in AI engineering — either overpaying for capability you don't need, or under-provisioning a task that genuinely needs a stronger model.

Why should AI Engineers learn it?

A professional AI engineer can justify a model choice with evidence (capability needs, cost at scale, context requirements, tool support) rather than "it's what I always use."

Where is it used in the real world?

Every production AI system starts with a model selection decision, and many use different models for different sub-tasks (Module 01, 3.12) — a cheap, fast model for simple routing, a stronger model for complex reasoning.

Why is it relevant today?

The model landscape changes fast — new versions, new pricing, new capabilities ship constantly across every provider. This module teaches the evaluation framework; you should always verify current specifics (pricing, exact context limits, latest benchmarks) against each provider's live documentation rather than trusting any static snapshot for long.

> **Warning**
> Specific pricing, context window sizes, and benchmark rankings change frequently across all providers. This module focuses on each family's general positioning and philosophy — always check current official documentation for exact, up-to-date numbers before making a production decision.

2. Before You Begin

RECAP

Model capabilities & limitations (Module 01, 3.13) — the general strong/weak framework you learned there applies again here, now comparing across providers instead of within one.

RECAP

Cost optimization (Module 09, 3.4) — model choice is one of the biggest levers in cost optimization; this module gives you the landscape to make that choice from.

Tip

When comparing models for a real project, test your actual task against your actual evaluation set (Module 09) rather than relying purely on general reputation — real performance on your specific task is what matters.

3. Main Lesson

Each model family below covers: **Definition → Simple Explanation → Analogy → Strengths → Weaknesses → Best Use Cases → Common Mistakes → Summary.** Specific pricing/context numbers are intentionally omitted — check current provider docs for those.

3.1 GPT (OpenAI)

DEFINITION

GPT is OpenAI's flagship LLM family, one of the earliest widely-adopted commercial LLM lines and the model behind ChatGPT.

SIMPLE EXPLANATION

GPT models are broadly capable generalists with a huge existing ecosystem of tools, tutorials, and third-party integrations built around them, reflecting their early and continued market presence.

> 💡 *Like an established, widely-adopted operating system — not necessarily "best" at every single thing, but backed by the largest ecosystem of existing tools and community knowledge.*

STRENGTHS

Broad general capability, large third-party ecosystem and tooling support, extensive community documentation and examples.

WEAKNESSES

As with any model family, specific weaknesses shift with each new release — always benchmark against your specific task rather than assuming category-wide strengths apply uniformly.

BEST USE CASES

General-purpose products where broad ecosystem/tooling compatibility is valuable, and teams already invested in OpenAI's tooling.

COMMON MISTAKES

Defaulting to GPT purely out of familiarity without benchmarking alternatives for a specific task's actual needs.

SUMMARY

GPT is a broadly capable, ecosystem-rich model family — a strong general default, worth comparing against alternatives for specialized needs.

3.2 Claude (Anthropic)

DEFINITION

Claude is Anthropic's LLM family, built with a strong emphasis on safety, careful instruction-following, and — relevant throughout this course — particularly reliable adherence to structured prompts like XML tags (Module 02, 3.5).

SIMPLE EXPLANATION

Claude is frequently noted for strong performance on coding tasks, careful reasoning, long-document handling, and following nuanced, detailed instructions closely.

> 💡 *Like a meticulous senior engineer who reads the full spec carefully before starting — strong at following detailed instructions precisely and reasoning carefully through complex, structured tasks.*

STRENGTHS

Strong coding capability, careful instruction-following, particularly strong adherence to

XML-structured prompts, large context windows well-suited to long-document work.

WEAKNESSES

As with all families, relative weaknesses shift release to release — benchmark against your specific task.

BEST USE CASES

Coding assistants and agents (Claude Code), long-document analysis, tasks requiring careful, nuanced instruction-following, and enterprise use cases valuing a strong safety posture.

COMMON MISTAKES

Not taking advantage of Claude's strong XML-tag following (Module 02, 3.5) when structuring prompts, missing an easy reliability win specific to this family.

SUMMARY

Claude emphasizes careful instruction-following, strong coding ability, and safety — a strong choice for structured, detail-sensitive, or long-context tasks.

3.3 Gemini (Google)

DEFINITION

Gemini is Google's LLM family, deeply integrated with Google's broader ecosystem (Search, Workspace, Cloud) and known for strong native multimodal capability.

SIMPLE EXPLANATION

Gemini models are frequently highlighted for multimodal understanding (text, image, audio, video together) and tight integration with Google Cloud infrastructure and products.

> 💡 *Like a product deeply woven into an existing suite of tools — its biggest advantage often isn't any single capability in isolation, but how naturally it plugs into everything else in that ecosystem.*

STRENGTHS

Strong native multimodal capability, tight integration with Google Cloud and Workspace products, competitive context window sizes.

WEAKNESSES

Relative strengths/weaknesses shift release to release — benchmark for your specific task.

BEST USE CASES

Multimodal applications (combining text with image/video/audio understanding), and teams already built on Google Cloud infrastructure.

COMMON MISTAKES

Overlooking Gemini's multimodal strengths when a task genuinely involves non-text input, defaulting to a text-only model unnecessarily.

SUMMARY

Gemini stands out for multimodal capability and Google ecosystem integration — a strong fit when a task spans beyond pure text.

3.4 Llama (Meta)

DEFINITION

Llama is Meta's LLM family, notable primarily for being openly released — allowing self-hosting, fine-tuning, and full control, unlike closed API-only models.

SIMPLE EXPLANATION

Because Llama's weights are open, teams can run it on their own infrastructure, fine-tune it on private data, and avoid per-token API costs entirely (in exchange for hosting/infrastructure costs instead).

> 💡 *Like the difference between renting a fully-serviced apartment (closed API models) versus buying a house you can renovate however you want (open-weight models) — more control and customization, but you handle the maintenance.*

STRENGTHS

Open weights enable self-hosting, full fine-tuning control, no per-token API dependency, and strong community/ecosystem around open deployment tooling.

WEAKNESSES

Requires your own infrastructure and ML ops expertise to self-host well; raw capability compared to top closed models varies by version and size.

BEST USE CASES

Organizations with data residency/privacy requirements needing on-premises deployment, heavy fine-tuning needs, or wanting to avoid per-token API cost at very high volume.

COMMON MISTAKES

Underestimating the infrastructure and ML ops investment required to self-host and maintain an open-weight model well compared to using a managed API.

SUMMARY

Llama's core differentiator is openness — full control and self-hosting flexibility, at the cost of taking on infrastructure responsibility yourself.

🌬️3.5 Mistral

DEFINITION

Mistral is a European AI company producing both open-weight and API-based LLMs, generally positioned around efficiency — strong performance relative to model size.

SIMPLE EXPLANATION

Mistral models are frequently noted for being smaller and faster while remaining competitive in capability, making them attractive when latency and cost efficiency matter alongside quality.

> 💡 *Like a compact car engineered for excellent fuel efficiency without sacrificing too much performance — not always the biggest or flashiest, but well-optimized for its size class.*

STRENGTHS

Efficient performance-to-size ratio, offers both open and API-based options, generally competitive latency.

WEAKNESSES

Smaller ecosystem/tooling compared to the largest providers; relative capability varies by specific model version.

BEST USE CASES

Latency-sensitive applications, cost-conscious deployments at scale, and teams wanting European data residency options.

COMMON MISTAKES

Assuming smaller/faster always means meaningfully weaker — evaluate on your actual task rather than assuming size alone predicts quality.

SUMMARY

Mistral emphasizes efficiency — strong performance-per-cost and speed, with both open

and API deployment flexibility.

3.6 Qwen (Alibaba)

DEFINITION

Qwen is Alibaba's LLM family, offered both as open-weight models and via API, with particular strength in multilingual capability, especially Chinese-English performance.

SIMPLE EXPLANATION

Qwen models are frequently highlighted for strong performance across multiple languages and for offering a range of model sizes, from small efficient variants to large flagship models.

> 💡 *Like a translator raised bilingually from childhood, versus one who learned a second language later — native-level fluency in more than one language is a genuine, distinct strength, not just a side feature.*

STRENGTHS

Strong multilingual capability (particularly Chinese-English), open-weight availability across a range of model sizes, competitive coding performance in recent versions.

WEAKNESSES

Ecosystem and English-language community documentation may be less extensive than the largest US-based providers.

BEST USE CASES

Multilingual products, especially those serving Chinese-language markets, and teams wanting open-weight flexibility (Section 3.4) with strong multilingual performance.

COMMON MISTAKES

Overlooking Qwen for multilingual products purely due to unfamiliarity, when it may be a genuinely strong fit for that specific need.

SUMMARY

Qwen's standout strength is multilingual capability combined with open-weight flexibility — worth strong consideration for multilingual products.

3.7 DeepSeek

DEFINITION

DeepSeek is a Chinese AI lab known for releasing highly capable open-weight models, notably making waves for achieving strong reasoning benchmark performance at a fraction of the typical training/inference cost of comparable models.

SIMPLE EXPLANATION

DeepSeek's models (particularly its reasoning-focused releases) demonstrated that very strong reasoning performance doesn't necessarily require the largest possible model or cost — a notable industry moment for cost-efficient capability.

> 💡 *Like a scrappy competitor showing up with race results that rival much larger, better-funded teams — proof that clever engineering can substitute for sheer scale in specific areas.*

STRENGTHS

Strong reasoning capability relative to cost, open-weight availability enabling self-hosting, notably efficient training/inference approaches.

WEAKNESSES

As with any newer entrant, ecosystem maturity and long-term support track record are still developing relative to more established providers.

BEST USE CASES

Cost-sensitive applications needing strong reasoning capability, and teams comfortable with open-weight self-hosting wanting a highly capable, efficient option.

COMMON MISTAKES

Assuming "newer and cheaper" automatically means "less capable" — DeepSeek specifically challenged that assumption on reasoning benchmarks and is worth real evaluation, not dismissal.

SUMMARY

DeepSeek demonstrated that strong reasoning capability can come at notably lower cost — a compelling option for cost-sensitive, reasoning-heavy tasks.

4. Visual Learning

🌳 Open-Weight vs. Closed API Models

```
Model Access Models
├── Closed / API-only (GPT, Claude, Gemini)
│   └── Managed, no infra needed, per-token pricing
└── Open-weight (Llama, Mistral, Qwen, DeepSeek — often both)
    └── Self-hostable, fine-tunable, infra required
```

📊 Evaluation Dimensions Framework

Dimension	Question to Ask
Capability	Does it perform well on tasks like mine, on my own evaluation set (Module 09)?
Context limits	Can it hold as much input as my task realistically needs (Module 01, 3.6)?
Pricing	What's the real cost at my expected scale (Module 09, 3.4)?
Tool support	Does it support the tool-use/structured-output features I need (Modules 04, 06)?
Access model	Do I need open-weight self-hosting, or is a managed API fine?

🧭 Rough Positioning Map

```
Multilingual focus      ——— Qwen
Multimodal focus        ——— Gemini
Coding / instruction-   ——— Claude
  following focus
Ecosystem breadth        ——— GPT
Efficiency / size ratio ——— Mistral
Open self-hosting         ——— Llama
Cost-efficient reasoning ——— DeepSeek

(General positioning — always verify against current benchmarks for your task)
```

5. Real World Applications

Use Case	Common Model Family Fit
Coding assistant / agent	Claude (strong coding + instruction-following)
Multimodal document/video analysis	Gemini (native multimodal strength)
On-premises deployment, data residency requirements	Llama, Mistral, or Qwen (open-weight options)
Multilingual customer support (Chinese-English)	Qwen
Cost-sensitive, reasoning-heavy batch processing	DeepSeek or a smaller-tier model from any provider

6. Practical Examples

Beginner: Matching task to family

For each of these tasks, name which model family from Section 3 you'd consider first, and why: (1) a bilingual Chinese-English support bot, (2) a video-content analysis tool, (3) a self-hosted internal coding assistant.

Beginner: Open vs. closed tradeoffs

List 3 scenarios where open-weight self-hosting (Llama, Mistral, Qwen, DeepSeek) would be preferable to a closed API, and 3 where a closed API would be preferable.

Intermediate: Building an evaluation comparison

Using your Module 09 evaluation harness, run the same test set against 2 different model families (where you have access) and compare results.

Advanced: Multi-model architecture design

Design a system (on paper) that uses different model families for different sub-tasks based on their strengths — justify each choice.

7. Code Examples

🐍Python — Provider-agnostic comparison harness sketch

```python
# pip install anthropic openai  (example: comparing two providers)
import anthropic
import openai

anthropic_client = anthropic.Anthropic(api_key="ANTHROPIC_KEY")
openai_client = openai.OpenAI(api_key="OPENAI_KEY")

test_set = [
    ("Summarize: 'The meeting is moved to 3pm Friday.'", "meeting moved to 3pm Friday"),
    # ... more test cases from your Module 09 evaluation set
]

def call_claude(prompt):
    r = anthropic_client.messages.create(
        model="claude-sonnet-4-6", max_tokens=100,
        messages=[{"role": "user", "content": prompt}]
    )
    return r.content[0].text

def call_gpt(prompt):
    r = openai_client.chat.completions.create(
        model="gpt-4o", max_tokens=100,
        messages=[{"role": "user", "content": prompt}]
    )
    return r.choices[0].message.content

# Run the same test set through both, using your Module 09 evaluation logic
for prompt, expected in test_set:
    print("Claude:", call_claude(prompt))
    print("GPT:", call_gpt(prompt))
```

🟨JavaScript — Simple cost comparison calculator

```javascript
// Rough cost comparison across hypothetical per-million-token prices
// ALWAYS check current official pricing pages — these numbers go stale fast
const models = [
  { name: "Model A (example)", inputPricePerM: 3, outputPricePerM: 15 },
  { name: "Model B (example)", inputPricePerM: 0.5, outputPricePerM: 1.5 },
];

function monthlyCost(model, avgInputTokens, avgOutputTokens, callsPerDay) {
  const dailyIn = (avgInputTokens / 1_000_000) * model.inputPricePerM * callsPerDay;
  const dailyOut = (avgOutputTokens / 1_000_000) * model.outputPricePerM * callsPerDay;
  return (dailyIn + dailyOut) * 30;
}

for (const model of models) {
  const cost = monthlyCost(model, 500, 150, 50000);
  console.log(`${model.name}: $${cost.toFixed(2)}/month at 50k calls/day`);
}
```

> **Tip**
>
> Build a reusable, provider-agnostic evaluation harness (extending Module 09's) so comparing a new model family later is just a matter of adding one new call function, not rebuilding your whole testing setup.

8. Common Mistakes

- **Defaulting to whichever model is most familiar**, without benchmarking alternatives for the specific task.
- **Trusting stale pricing or benchmark numbers** instead of checking current official documentation.
- **Assuming smaller/cheaper always means meaningfully weaker**, ignoring efficiency-focused options like Mistral or DeepSeek.
- **Overlooking open-weight options** when data residency, fine-tuning, or infrastructure control genuinely matter.
- **Choosing based on general reputation alone**, rather than testing against your own evaluation set (Module 09).
- **Ignoring multimodal or multilingual needs** when they're actually relevant to the task.

9. Best Practices

- **Evaluate against your own task and test set** (Module 09), not just general reputation.
- **Check current official docs** for pricing, context limits, and capabilities before deciding — this landscape changes fast.
- **Match model choice to genuine task needs**: multimodal, multilingual, coding, cost sensitivity, or data residency.
- **Consider mixing models** across sub-tasks within one system, rather than assuming one model must handle everything.
- **Weigh open-weight vs. closed API tradeoffs** honestly, including the real infrastructure cost of self-hosting.

10. Hands-on Activities

Easy 5 exercises

1. List each of the 7 model families and one distinguishing strength for each, from memory.

2. For a fictional multilingual product, explain which model family you'd evaluate first and why.

3. Explain the core tradeoff between open-weight and closed API models in your own words.

4. Visit one provider's official pricing page and note today's current pricing for their flagship model.

5. List 3 questions you'd ask before choosing a model for a new project (from the Section 4 framework).

Intermediate 3 exercises

1. Build the provider-agnostic evaluation harness sketch from Section 7 with 2 real providers you have API access to.

2. Run the same 5-question test set through 2 different model families and compare answers, cost, and latency.

3. Estimate monthly cost for a realistic call volume using 2 different models, following the Section 7 JS example.

Challenge 2 exercises

1. Design a multi-model system architecture (on paper) that routes different sub-tasks to different model families based on their strengths, with justification for each routing decision.

2. Research current benchmark leaderboards for 3 of the 7 families and write a short summary of how the landscape has shifted since this module was written.

11. Mini Project

Goal

Build a small model comparison tool that runs the same test set (from Module 09) against
2+ model providers and reports accuracy, cost, and latency side by side.

Requirements

- API keys for at least 2 different model providers
- Python or JavaScript
- A test set with known expected outputs (reuse or extend your Module 09 test set)

Step-by-Step Instructions

1. Write a wrapper function per provider that takes a prompt and returns the response, token usage, and response time.
2. Run your full test set through each provider.
3. Score each provider's accuracy using your Module 09 evaluation logic.
4. Compute average cost per call and average latency for each provider.
5. Print a comparison table: accuracy, cost, latency, side by side.

Expected Output

```
Provider    Accuracy   Avg Cost/Call   Avg Latency
Claude       90%          $0.0021          1.2s
GPT          85%          $0.0018          0.9s

Verdict: Claude is more accurate but slightly pricier and slower for this task.
```

Possible Improvements

- Add a third provider (open-weight, self-hosted or via an aggregator) to the comparison.
- Weight the comparison by your project's actual priorities (e.g., accuracy matters more than cost for this task).
- Automate this comparison to re-run periodically as models update.

12. Review Questions

Multiple Choice (10)

1. Which is a key differentiator of open-weight models like Llama?

A) They can't be self-hosted B) They can be self-hosted and fine-tuned without per-token API dependency C) They have no strengths D) They are always the most capable

2. Gemini is particularly known for:

A) Being text-only B) Strong native multimodal capability C) Being the only open-weight model D) Having no API access

3. Claude is frequently noted for strength in:

A) Multimodal video processing only B) Coding and careful instruction-following C) Being open-weight D) Having no context window

4. Qwen's standout strength is generally:

A) Multilingual capability, especially Chinese-English B) Being closed-source only C) Having no coding ability D) Being the most expensive option

5. DeepSeek notably demonstrated:

A) That scale is the only path to capability B) Strong reasoning performance at notably lower cost C) That open models are always worse D) A completely new modality

6. Mistral is generally positioned around:

A) Maximum possible size B) Efficiency — strong performance relative to size/cost C) Closed-source only access D) Multimodal-only capability

7. GPT's key advantage is often described as:

A) Being the only model that exists B) Broad capability plus a large ecosystem/tooling base C) Open weights D) No API access

8. Why should you check official docs before a production model decision?

A) Pricing and capabilities change frequently across providers B) It's never necessary C) Documentation is always wrong D) Only for open-weight models

9. What's the recommended way to compare models for your specific task?

A) Trust general reputation only B) Test against your own evaluation set (Module 09) C) Always pick the newest release D) Always pick the cheapest option regardless of task

10. A key tradeoff of open-weight self-hosting is:

A) No tradeoffs exist B) Infrastructure/ML ops responsibility shifts to you C) It's always cheaper with no effort D) It removes the need for evaluation

Identification (5)

1. _____ is Anthropic's LLM family, known for careful instruction-following and strong coding capability.

2. _____ is Google's LLM family, known for strong native multimodal capability.

3. _____ is Meta's open-weight LLM family.

4. _____ is known for strong multilingual (especially Chinese-English) capability.

5. _____ demonstrated strong reasoning performance at notably lower training/inference cost.

True or False (5)

1. Open-weight models can be self-hosted and fine-tuned without depending on a per-token API.

2. Model pricing and capabilities are static and never change once published.

3. A smaller or cheaper model is always meaningfully less capable than a larger one.

4. Different model families can be mixed within a single system, with each handling the sub-tasks it's best suited for.

5. Testing against your own evaluation set is a more reliable way to choose a model than general reputation alone.

Situational (5)

1. You need a coding agent with strong instruction-following for a complex refactoring tool. Which family would you evaluate first?

2. Your product needs to analyze both video and text content together. Which family's strength matches this need?

3. Your company has strict data residency requirements requiring on-premises deployment. What category of model should you look at?

4. You're building a Chinese-English bilingual support bot. Which family would you evaluate first?

5. You need strong reasoning performance but have a tight budget. Which family notably challenged the assumption that reasoning capability requires high cost?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-A, 5-B, 6-B, 7-B, 8-A, 9-B, 10-B

Identification: 1-Claude, 2-Gemini, 3-Llama, 4-Qwen, 5-DeepSeek

True or False: 1-True, 2-False, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Claude — noted strength in coding and careful instruction-following.
2. Gemini — strong native multimodal capability.
3. Open-weight models (Llama, Mistral, Qwen, or DeepSeek) for on-premises self-hosting.
4. Qwen — strong Chinese-English multilingual capability.
5. DeepSeek — demonstrated strong reasoning at notably lower cost.

13. Cheat Sheet

🔒 Closed / API-Only

GPT — broad ecosystem.
Claude — coding, instruction-following.
Gemini — multimodal, Google integration.

🔓 Open-Weight (often both)

Llama — openness, self-hosting.
Mistral — efficiency.
Qwen — multilingual.
DeepSeek — cost-efficient reasoning.

📊 Evaluate On

Capability (your test set), context limits, pricing, tool support, access model (open vs. closed).

⚠ Always Remember

Specifics change fast — verify pricing/benchmarks against current official docs before deciding.

14. Key Takeaways

- Model choice is a deliberate engineering decision, evaluated on capability, context, cost, and tool support for your specific task.
- GPT offers broad capability and ecosystem breadth; Claude excels at coding and careful instruction-following; Gemini leads in native multimodal capability.
- Llama, Mistral, Qwen, and DeepSeek offer open-weight flexibility, each with distinct strengths: self-hosting freedom, efficiency, multilingual capability, and cost-efficient reasoning respectively.
- Open-weight models trade API convenience for infrastructure control and self-hosting responsibility.
- Always test against your own evaluation set (Module 09) rather than relying purely on general reputation.
- Pricing, benchmarks, and capabilities change fast across all providers — verify against current official documentation before production decisions.

15. Additional Resources

Official Documentation

- Anthropic — docs.claude.com (Claude models, pricing, capabilities)
- OpenAI — platform.openai.com/docs
- Google — ai.google.dev (Gemini)
- Meta — llama.com (Llama)
- Mistral AI — docs.mistral.ai
- Alibaba Cloud — Qwen official documentation
- DeepSeek — official documentation and papers

Tools

- Independent benchmark leaderboards (check multiple sources, as methodology varies) for current comparative performance.
- OpenRouter and similar aggregators for testing multiple model families through one unified API during evaluation.

Communities & Further Learning

- Provider release notes and changelogs — the fastest way to stay current on this fast-moving landscape.